

Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

Disciplina: Teste e Manutenção de Software

Professor: Pedro Felipe

Trabalho Prático 2

Casos de Teste e Cobertura de Código

3S API — Sistema de Gestão de Pedidos, Aluguéis e Contratos
--

Grupo: 3S API

Integrantes:

Bruna Luiza Siqueira Borges Matias

Gabriel Baumgratz de Paula Botaro

Geovane Araujo Viana

João Pedro Ferreira Bonifácio

Ricardo Araújo Maciel

Samuel Maia de Oliveira

Vitor Fortunato Silva

Belo Horizonte, 2026

1. DESCRIÇÃO DO SOFTWARE

A **3S API** é um sistema de gestão empresarial desenvolvido para a empresa *3S Letreiros*, especializada na produção e locação de letreiros e placas decorativas. A aplicação é uma API REST completa construída com **Java 21** e **Spring Boot 4.0.1**, utilizando banco de dados **PostgreSQL** hospedado na nuvem (Neon), autenticação via **JWT** e comunicação em tempo real por **WebSocket (STOMP)**.

O sistema centraliza as operações de negócio da empresa em um único back-end, expondo endpoints REST consumidos por um front-end web. As principais funcionalidades são:

- **Gestão de usuários:** cadastro, autenticação com JWT + refresh token, atualização de perfil
- **Gestão de produtos:** controle de estoque, preço, imagem e busca por nome
- **Gestão de clientes:** cadastro com validação de CPF e e-mail únicos
- **Pedidos de venda:** criação com itens, cálculo de total, controle de status e devolução de estoque no cancelamento
- **Aluguéis:** reserva de produtos com validação de disponibilidade no período, taxa de entrega e múltiplos status
- **Contratos:** geração de contratos em PDF com cláusulas customizáveis, assinatura e cancelamento
- **Recibos:** geração de recibos em PDF para pedidos e aluguéis
- **Dashboard gerencial:** métricas de aluguéis ativos, receita mensal e últimas transações
- **Notificações em tempo real:** eventos publicados via WebSocket após cada operação crítica

A arquitetura segue o padrão de **três camadas**: *Presentation* (Controllers e DTOs), *Domain* (regras de negócio, entidades, serviços e políticas) e *Infrastructure* (persistência JPA, segurança, configurações).

2. ESTRUTURA DO SOFTWARE

A tabela abaixo resume a estrutura quantitativa do projeto:

Camada	Classes / Arquivos	Métodos / Funções
Domain (regras de negócio)	~55	~130
Infrastructure (JPA, segurança)	~20	~60
Presentation (controllers, DTOs)	~20	~45
Total do projeto	~95	~235

O escopo dos testes unitários foi a **camada de domínio**, que contém toda a lógica de negócio da aplicação. As camadas de infraestrutura e apresentação requerem testes de integração com banco de dados real, fora do escopo deste trabalho. No escopo medido pelo JaCoCo, foram analisados **69 classes** com **183 métodos**.

3. CASOS DE TESTE DESENVOLVIDOS

Foram desenvolvidos **169 casos de teste** distribuídos em **31 suítes de teste** (arquivos *.java* com sufixo `Test`), todas aprovadas com **0 falhas**. Os testes foram escritos utilizando **JUnit 5 (Jupiter)** e **Mockito 5** para isolamento de dependências.

Suíte de Teste	Classe Testada	Nº Testes	Resultado
CpfTest	Cpf (Value Object)	12	✓ OK
AddressTest	Address (Value Object)	3	✓ OK
ProductTest	Product (Entidade)	7	✓ OK
OrderItemTest	OrderItem (Entidade)	5	✓ OK
OrderItemPolicyTest	OrderItemPolicy	7	✓ OK
RentItemTest	RentItem (Entidade)	5	✓ OK
RentItemPolicyTest	RentItemPolicy	5	✓ OK
OrderTest	Order (Entidade)	8	✓ OK
OrderStatusTest	OrderStatus (Enum)	10	✓ OK
RentTest	Rent (Entidade)	7	✓ OK
RentStatusTest	RentStatus (Enum)	9	✓ OK
ContractTest	Contract (Entidade)	10	✓ OK
RefreshTokenPolicyTest	RefreshTokenPolicy	5	✓ OK
UserEntityTest	User (Entidade)	10	✓ OK
UserPolicyTest	UserPolicy	6	✓ OK
CostumerTest	Costumer (Entidade)	6	✓ OK
CreateUserServiceTest	CreateUserService	5	✓ OK
LoginUserServiceTest	LoginUserService	5	✓ OK
GetUserServiceTest	GetUserService	2	✓ OK
UpdateUserServiceTest	UpdateUserService	2	✓ OK
CreateProductServiceTest	CreateProductService	3	✓ OK
GetProductsServiceTest	GetProductsService	5	✓ OK
UpdateProductServiceTest	UpdateProductService	2	✓ OK
DeleteProductServiceTest	DeleteProductService	1	✓ OK

Suíte de Teste	Classe Testada	Nº Testes	Resultado
GetOrderServiceTest	GetOrderService	5	✓ OK
UpdateOrderServiceTest	UpdateOrderService	4	✓ OK
GetRentServiceTest	GetRentService	5	✓ OK
GetCostumerServiceTest	GetCostumerService	6	✓ OK
UpdateContractServiceTest	UpdateContractService	4	✓ OK
CreateClauseServiceTest	CreateClauseService	2	✓ OK
GetClauseServiceTest	GetClauseService	2	✓ OK
TOTAL	31 suítes	169	169 ✓

4. CÁLCULO DE COBERTURA DE TESTES

4.1 Ferramenta Utilizada

A cobertura foi calculada automaticamente pela ferramenta **JaCoCo 0.8.12** (*Java Code Coverage*), integrada ao projeto via plugin Maven. O relatório é gerado automaticamente ao executar o comando `mvn test`, sendo salvo em `target/site/jacoco/` nos formatos HTML, XML e CSV.

4.2 Escopo da Medição

O escopo de medição foi restrito à **camada de domínio** do projeto. As camadas de infraestrutura (persistência JPA, segurança, configurações) e de apresentação (controllers REST, DTOs) foram excluídas por serem adaptadores técnicos que não contêm regras de negócio e requerem testes de integração com banco de dados real para serem validados adequadamente. Essa é uma prática padrão em arquiteturas que seguem os princípios de separação de responsabilidades.

4.3 Resultados Obtidos

Resultado geral: 71,06% de cobertura de linhas | 169 testes aprovados | 0 falhas

Métrica	Cobertas	Não Cobertas	Total	Cobertura
Linhas de código	474	193	667	71,06%
Métodos / funções	128	55	183	69,95%
Classes testadas	~50	~19	69	~72%

4.4 Cobertura por Classe (principais)

Classe	Tipo	Linhas Cobertas	Total	%
Cpf	Value Object	13	13	100%
Address	Value Object	1	1	100%
User	Entidade	55	55	100%
Costumer	Entidade	21	21	100%
Product	Entidade	22	22	100%
Order	Entidade	22	22	100%
OrderStatus	Enum	19	19	100%

Classe	Tipo	Linhas Cobertas	Total	%
OrderItem	Entidade	11	11	100%
OrderItemPolicy	Política	6	6	100%
Rent	Entidade	23	23	100%
RentStatus	Enum	20	20	100%
RentItem	Entidade	11	11	100%
RentItemPolicy	Política	6	6	100%
Contract	Entidade	30	30	100%
UserPolicy	Política	9	9	100%
RefreshTokenPolicy	Política	3	3	100%
CreateUserService	Serviço	10	10	100%
LoginUserService	Serviço	9	9	100%
UpdateContractService	Serviço	14	14	100%
UpdateOrderService	Serviço	21	25	84%
Clause	Entidade	7	15	47%
Exceções de domínio (12 classes)	Exceção	24	48	50%

4.5 Como Reproduzir a Medição

Para reproduzir os testes e gerar o relatório de cobertura:

1. Clonar o repositório: `git clone [URL_DO_REPOSITORIO]`
2. Acessar o diretório: `cd api`
3. Executar: `mvn test` (requer Java 21 e Maven 3.9+)
4. Abrir o relatório: `target/site/jacoco/index.html`

5. CONSIDERAÇÕES FINAIS

5.1 Estratégia de Teste Adotada

A equipe adotou a estratégia de **testes unitários** com foco na camada de domínio, por ser a camada que concentra toda a lógica de negócio da aplicação e é independente de frameworks externos (banco de dados, HTTP, WebSocket). Isso permite que os testes sejam rápidos, determinísticos e fáceis de manter.

Para os serviços que possuem dependências externas (repositórios, template de mensagens), foi utilizado **Mockito** para criar objetos simulados (*mocks*), garantindo o isolamento da unidade testada. Essa técnica permite verificar não apenas o resultado das operações, mas também se as dependências foram chamadas corretamente (verificação de interações via `verify()`).

5.2 Tipos de Casos de Teste Implementados

- **Testes de valor válido (happy path):** verificam o comportamento correto quando os dados de entrada são válidos
- **Testes de valor inválido:** verificam se exceções são lançadas com dados inválidos (ex.: CPF com menos de 11 dígitos, quantidade de item igual a zero)
- **Testes de valor limite:** verificam comportamento nos limites (ex.: quantidade exatamente igual ao estoque disponível)
- **Testes de estado:** verificam transições de estado das entidades (ex.: contrato cancelado não pode ser assinado)
- **Testes de interação:** verificam se os colaboradores (repositórios, serviços) foram invocados com os parâmetros corretos
- **Testes de nulidade:** verificam comportamento com valores nulos (ex.: cálculo de subtotal com valor unitário nulo)

5.3 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Java	21	Linguagem de programação
Spring Boot	4.0.1	Framework da aplicação

Tecnologia	Versão	Finalidade
JUnit 5 (Jupiter)	5.13.1	Framework de testes unitários
Mockito	5.x	Criação de mocks e verificação de interações
JaCoCo	0.8.12	Medição de cobertura de código
Maven	3.9+	Gerenciamento de build e dependências
PostgreSQL	—	Banco de dados (produção)

5.4 Repositório do Projeto

O código-fonte completo, incluindo todos os casos de teste implementados, está disponível em:

GitHub: <https://github.com/samuelmaia1/3s-api>

5.5 Conclusão

O trabalho demonstrou a importância e a viabilidade de se obter alta cobertura de testes em sistemas reais de médio porte. Com 169 casos de teste e cobertura de **71,06% das linhas** de código da camada de domínio, superou-se o requisito mínimo de 70% estabelecido para o trabalho. A adoção de ferramentas automatizadas como JaCoCo elimina subjetividade no cálculo da cobertura e fornece relatórios detalhados que facilitam a identificação de áreas de risco não testadas.